

Logistic Regression — Code Explanation

Detailed Step-by-Step Technical Analysis of `logistic_regression.py`

Author: Aymen Negadi | **Script File:** `logistic_regression.py` | **Framework:** Scikit-Learn / Python 3

1. Module Imports

The code begins by importing all required analytical and machine learning dependencies:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

- `numpy` & `pandas` handle data manipulation, structure arrays, and construct tables.
- `load_breast_cancer` fetches the built-in diagnostic dataset.
- `train_test_split` splits the data randomly into training and testing partitions.
- `StandardScaler` performs z-score feature normalization.
- `LogisticRegression` is the classification algorithm class.
- Metrics functions evaluate model predictions against true labels.

2. Dataset Loading & Structural Exploration

```
cancer = load_breast_cancer()

print("Feature names:")
print(cancer.feature_names)

print("Number of samples:")
print(len(cancer.data))
print(cancer.data.shape)
print("First 10 diagnoses:")
print(cancer.target[:10])
```

Detailed Explanation:

- `load_breast_cancer()` returns a dictionary-like object containing data arrays, feature names, target arrays, and description.
- `cancer.feature_names` outputs an array of 30 string names representing anatomical measurements (e.g., 'mean radius', 'mean texture').
- `cancer.data.shape` outputs `(569, 30)`, confirming 569 sample rows and 30 feature columns.
- `cancer.target[:10]` displays the first 10 ground truth target values (e.g. 0 for malignant).

3. Constructing & Inspecting the DataFrame

```
df = pd.DataFrame(cancer.data, columns=cancer.feature_names)
df["Diagnosis"] = cancer.target

print(df.head())
print("
DataFrame Information:")
print(df.info())
print("
Diagnosis Count:")
print(df["Diagnosis"].value_counts())
```

Detailed Explanation:

- `pd.DataFrame(...)` converts the raw NumPy array into a structured tabular DataFrame with column headers.
- `df["Diagnosis"] = cancer.target` appends the target label column as the 31st column.
- `df.info()` checks memory footprint, data types (`float64` for features, `int64` for target), and confirms zero missing values.
- `df["Diagnosis"].value_counts()` counts class distribution (357 benign vs 212 malignant samples).

4. Feature-Target Separation & Dataset Partitioning

```
X = df.drop("Diagnosis", axis=1)
y = df["Diagnosis"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Detailed Explanation:

- `df.drop("Diagnosis", axis=1)` isolates the 30 input feature columns into `X`.
- `y` stores only the binary target column.
- `train_test_split(..., test_size=0.2, random_state=42)` partitions 80% (455 samples) into training sets and 20% (114 samples) into test sets. Setting `random_state=42` guarantees identical splits across runs.

5. Feature Standardization

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Detailed Explanation:

- `scaler.fit_transform(X_train)` calculates the mean (μ) and standard deviation (σ) of each feature from `X_train` and transforms `X_train` into z-scores.
- `scaler.transform(X_test)` applies the *exact same* training mean and standard deviation to standardize `X_test`. Crucially, we do NOT call `fit_transform` on test data to avoid data leakage.

6. Model Instantiation & Training

```
model = LogisticRegression()  
model.fit(X_train, y_train)
```

Detailed Explanation:

- `LogisticRegression()` initializes the binary logistic classifier using default optimization solver (`lbfgs`) and L2 regularization.
- `model.fit(...)` uses optimization algorithms to learn weight coefficients $w_1, w_2, \dots, w_{30}$ and bias b that minimize the binary cross-entropy loss function over the training data.

7. Predictions & Quantitative Evaluation

```
y_pred = model.predict(X_test)  
  
accuracy = accuracy_score(y_test, y_pred)  
cm = confusion_matrix(y_test, y_pred)  
report = classification_report(y_test, y_pred)
```

Detailed Explanation:

- `model.predict(X_test)` applies the sigmoid decision threshold ($P \geq 0.5 \rightarrow 1$) to classify each test sample.
- `accuracy_score` computes the proportion of correct predictions: $(TP + TN) / (TP + TN + FP + FN) = (70 + 41) / 114 = 97.37\%$.
- `confusion_matrix` generates the 2x2 matrix comparing actual vs predicted classes.
- `classification_report` calculates class-wise Precision, Recall, and F1-score.